

Processador SAP-1

Relatório de Implementação em Verilog (RTL)

*Arquitetura · Decisões de projeto · Temporização · Máquina de estados ·
Código-fonte*

*Plataforma-alvo: Terasic DE10-Lite (Intel MAX 10 10M50DAF484C7G) · Quartus
Prime Lite 20.1*

Disciplina: _____

Integrantes: _____

Data: ____ / ____ / ____

1. Introdução

Este relatório documenta a implementação em Verilog do processador SAP-1 (Simple-As-Possible): como o projeto está organizado, quais decisões de projeto foram tomadas — e por quê, no nível de hardware —, como a temporização foi disciplinada em duas bordas de clock, como a unidade de controle foi modelada por máquina de estados, e o código-fonte comentado. Serve de referência técnica do que foi construído, complementando o relatório de verificação por simulação.

Resumo das especificações implementadas:

Parâmetro	Valor	Onde
Largura de dados	8 bits	barramento W, A, B, ULA, saída
Largura de endereço	4 bits	PC e MAR (16 posições)
Memória	16 × 8 bits (ROM em execução)	ram_16x8.v
Registradores de dados	2 (acumulador A e B)	accumulator.v, register_b.v
ULA	soma e subtração, 8 bits	adder_subtractor.v
Instruções	5 (LDA, ADD, SUB, OUT, HLT)	decodificadas no controle
Estados de controle	6 (T1-T6) + IDLE de partida	controller_sequencer.v
Palavra de controle	12 bits (CON)	gerada a cada estado

2. Estrutura do projeto

O projeto separa o núcleo sintetizável (rtl) da camada específica de placa (fpga), da verificação (tb_model) e da documentação:

```
SAP1/  
|- rtl/          nucleo do processador (11 modulos, sinteticaveis, sem I/O de placa)  
|- fpga/         camada da placa DE10-Lite (top de placa, clock, debounce, displays)  
|- tb_model/     14 testbenches auto-verificaveis + scripts de onda (ModelSim)  
|- programas/   programas de teste em .txt (1..4) para colar na RAM  
|- fsm/         diagramas da maquina de estados (Python/matplotlib)  
|- docs/        documentacao e relatorios (este arquivo)  
|- apresentacao/ slides e imagens (datapath, ondas)  
|- sap1_top.qsf, SAP1.qpf projeto Quartus (pinos, arquivos)
```

A separação **rtl** × **fpga** é deliberada e tem efeito prático: o núcleo em **rtl** não conhece displays, botões nem o clock de 50 MHz da placa — suas únicas entradas são **clk** e **clr_bar**. Quem adapta o núcleo ao hardware real (divide o

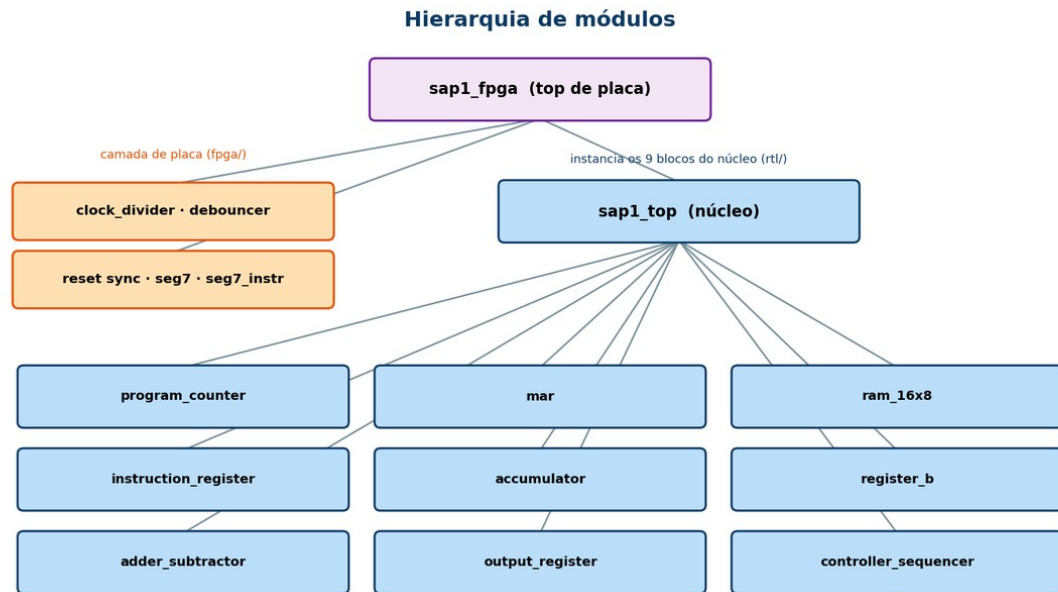
clock, filtra o botão, aciona os sete segmentos) é a pasta **fpga**. Assim, o mesmo núcleo é simulado e sintetizado sem alteração.

2.1. Módulos do núcleo (rtl)

Módulo	Papel	Natureza
sap1_top.v	Topo: barramento W (mux) + instâncias de todos os blocos	estrutural
program_counter.v	PC — contador de 4 bits do endereço de instrução	sequencial
mar.v	Registrador de endereço apresentado à memória	sequencial
ram_16x8.v	Memória 16×8, leitura combinacional (ROM em execução)	combinacional
instruction_register.v	IR — guarda a instrução e a fatia em opcode operando	sequencial
accumulator.v	Acumulador A (registrador de trabalho)	sequencial
register_b.v	Registrador B (segundo operando da ULA)	sequencial
adder_subtractor.v	ULA — soma/subtração em 8 bits (complemento de dois)	combinacional
output_register.v	Registrador de saída (visor)	sequencial
controller_sequencer.v	Controle usado: anel one-hot + palavra de controle	sequencial
controller_fsm.v	Controle alternativo: FSM clássica de 3 processos	sequencial

2.2. Módulos da camada de placa (fpga)

Módulo	Papel
sap1_fpga.v	Top de placa: junta núcleo, clock, reset, botões e displays
clock_divider.v	Divide 50 MHz para ~1 Hz (execução visível)
debouncer.v	Antirruído do botão (clock manual passo a passo)
seg7.v	Decodificador hexadecimal → 7 segmentos
seg7_instr.v	Decodificador opcode → letra (L/A/5/o/H)



controller_fsm.v é uma implementação ALTERNATIVA do controle (mesmas portas) — trocável no topo.

Figura 1 — Hierarquia: o top de placa instancia a camada fpga e o núcleo sap1_top, que por sua vez instancia os 9 blocos do datapath e controle.

A figura evidencia a separação de responsabilidades: o núcleo sap1_top (azul) apenas instancia os nove blocos do datapath e do controle, sem conhecer nada da placa; toda a adaptação ao hardware (clock, botão, displays) fica na camada fpga (laranja), sob o top de placa. Note ainda que controller_fsm.v é uma alternativa ao controller_sequencer.v — mesmas portas, trocável no topo.

3. Arquitetura e convenções de sinais

Datapath do SAP-1

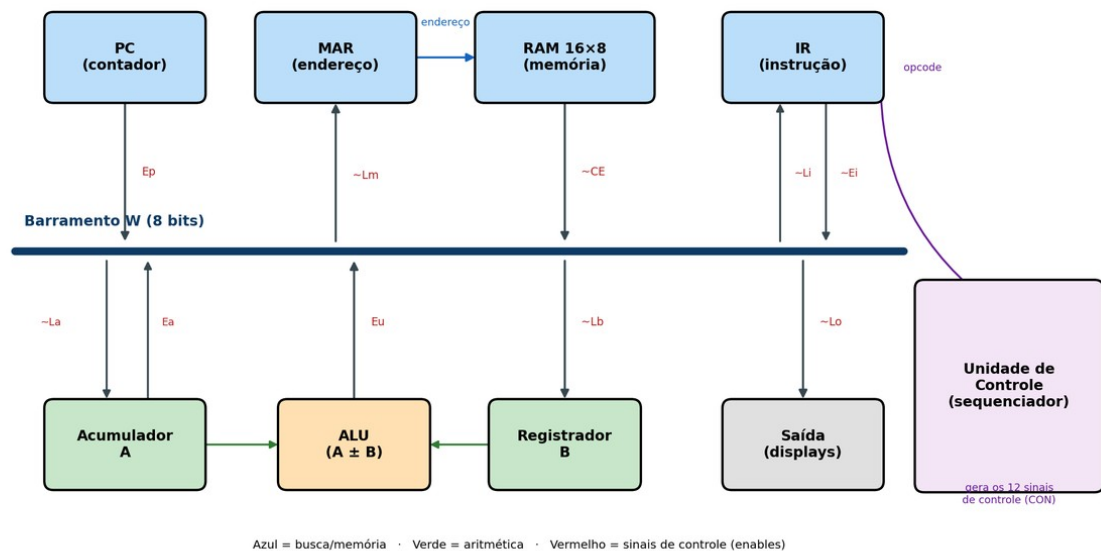


Figura 2 — Datapath do SAP-1: barramento W, blocos de dados e unidade de controle.

Todos os blocos se comunicam por um único barramento de 8 bits (barramento W). A regra de uso do barramento é rígida e é o que garante a coerência do datapath:

- **Em cada estado, no máximo um bloco escreve** no barramento (um único "enable" de saída ativo); um ou mais blocos podem **ler** simultaneamente.
- Fontes de 4 bits (PC e operando do IR) são **estendidas com zeros** à esquerda para 8 bits: {4'b0000, pc_out}. O MAR, por sua vez, lê apenas os 4 bits baixos do barramento (w_bus[3:0]).

3.1. Convenção de nomes e níveis ativos

A nomenclatura segue a tradição do SAP-1 e é seguida à risca no código, o que evita erros de polaridade:

- Sinais **ativos em alto** (1 = ativo): Cp, Ep, Ea, Su, Eu. São "habilitações" de contagem/saída no barramento.
- Sinais **ativos em baixo** (0 = ativo), grafados com til/sufixo _bar: ~Lm, ~CE, ~Li, ~Ei, ~La, ~Lb, ~Lo. São as cargas de registradores e a habilitação da RAM.

Consequência importante: o **repouso não é tudo-zero**. Como as cargas são ativas em baixo, a palavra de repouso precisa manter esses bits em 1. Por isso a palavra ociosa é **12'b0011_1110_0011**, e não zero.

4. Disciplina de temporização em duas bordas

A decisão de temporização mais estrutural do projeto é usar as duas bordas do clock, com papéis distintos:

- O **contador de estados avança na borda de descida** (negedge) do clock.
- Os **registradores do datapath carregam na borda de subida** (posedge).

O motivo é eliminar uma condição de corrida entre "decidir o que fazer" e "fazer". Ao trocar de estado no negedge, a nova palavra de controle já está estável e propagada quando chega o posedge seguinte — instante em que os registradores amostram o barramento. Se estado e dados mudassem na mesma borda, o registrador poderia amostrar valores da microoperação errada. Na prática, cada estado T_n ocupa meio período "preparando" o barramento e o posedge no meio dele efetiva a transferência. Essa relação é o que torna previsível a leitura das ondas (o acumulador de um ADD só assenta no T_6 e aparece assentado já no T_1 seguinte).

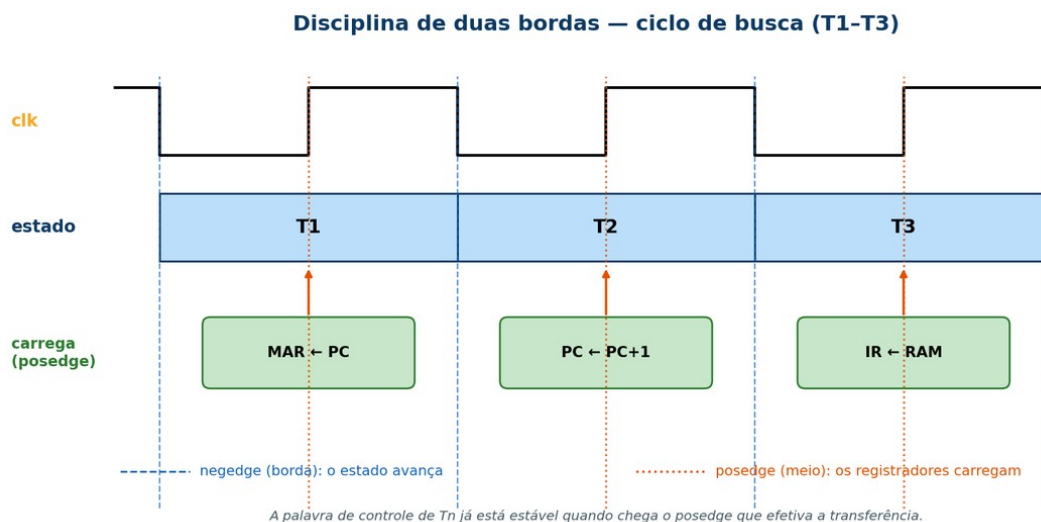


Figura 3 — Duas bordas: o estado avança no negedge (bordas) e os registradores carregam no posedge (meio de cada estado).

A figura resume essa disciplina para o ciclo de busca. Nas linhas azuis (bordas de descida) o estado muda — T1, T2, T3 —; nas linhas laranja (bordas de subida, no meio de cada estado) os registradores amostram o barramento: $MAR \leftarrow PC$ em T1, $PC \leftarrow PC+1$ em T2 e $IR \leftarrow RAM$ em T3. Como a palavra de controle de cada estado já está estável antes do posedge correspondente, não há ambiguidade sobre qual transferência ocorre.

5. Decisões de implementação

As decisões a seguir são ilustradas com trechos de código. As construções de Verilog empregadas (*assign*, *always*, *reg/wire*, *reset assíncrono*, *non-blocking*, etc.) estão resumidas na seção 10.1 — vale consultá-la antes se o Verilog não for familiar.

5.1. Barramento W por multiplexador com prioridade

Nos livros o barramento costuma ser desenhado com buffers tri-state. Optou-se por um multiplexador com cadeia de prioridade (if-else aninhado via operador ternário), pela razão concreta de que FPGAs da família MAX 10 não têm barramentos tri-state internos: a forma sintetizável de "vários blocos, um fio" é justamente o mux. A cadeia testa os enables em ordem; funcionalmente a ordem é irrelevante porque a unidade de controle garante exclusão mútua (só um enable ativo por estado). O ramo final (0000_0000) evita propagação de X na simulação quando, por construção, nenhum enable está ativo (estados de repouso).

rtl/sap1_top.v – multiplexador do barramento W

```
assign w_bus = ep      ? {4'b0000, pc_out}      : // Ep : PC (4b) -> 8b
                  (ce_bar==1'b0) ? ram_data      : // ~CE: RAM (8b)
                  (ei_bar==1'b0) ? {4'b0000, ir_operand} : // ~Ei: operando (4b) -> 8b
                  ea      ? acc_out              : // Ea : acumulador
                  eu      ? alu_out              : // Eu : saída da ULA
                  8'b0000_0000;                // repouso (nenhum fala)
```

5.2. Padrão de registrador: reset assíncrono + carga com prioridade

Cinco blocos (A, B, MAR, IR e saída) são o mesmo flip-flop com enable. O reset aparece na lista de sensibilidade (negedge clr_bar), tornando-o assíncrono — ele vence qualquer coisa. Em seguida, a carga só ocorre quando o sinal está ativo; se não houver carga, o bloco simplesmente não atribui, e a síntese infere o "segura o valor" de um flip-flop com clock-enable (não gera latch, pois todo caminho está sob a borda de clock). Padronizar isso deixou o núcleo uniforme — muda só a largura e o nome da carga.

rtl/accumulator.v – padrão de A, B, MAR, IR e saída

```
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)    acc_out <= 8'b0000_0000; // (1) reset assincrono, prioridade
    else if (!la_bar) acc_out <= bus_in;      // (2) carrega quando ~La = 0
end                                              // (3) senao: mantem (FF com enable)
```

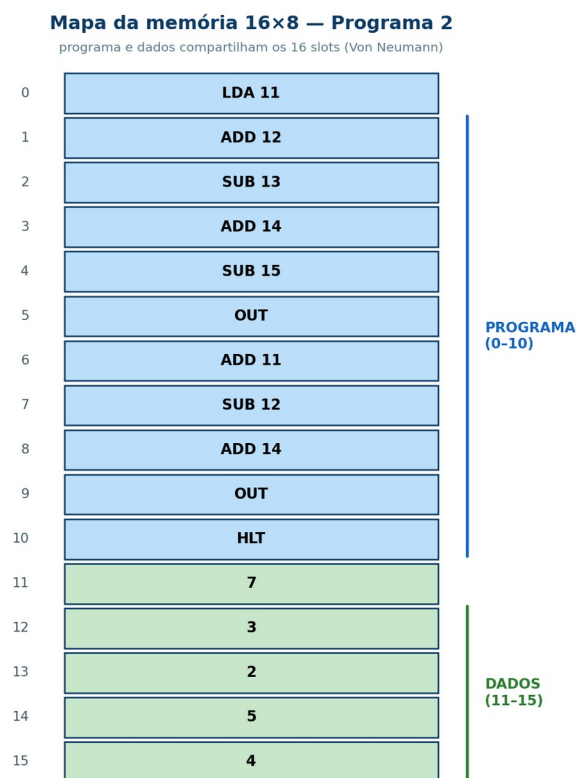
5.3. Memória como ROM inicializada (sem STA)

O SAP-1 básico não tem instrução de escrita (STA); logo a RAM é somente de leitura durante a execução. Um bloco inicial preenche os 16 endereços e a leitura é combinacional (*assign data_out = mem[addr]*), sem clock — um "asynchronous read ROM". Na síntese isso vira memória de inicialização (ROM). Como programa e dados dividem os mesmos 16 slots (Von Neumann), o tamanho do programa limita o espaço de dados. Trocar de programa =

editar este bloco e recompilar; por isso os quatro programas ficam prontos em programas/*.txt.

rtl/ram_16x8.v

```
reg [7:0] mem [0:15];
integer i;
initial begin
    for (i = 0; i < 16; i = i + 1) mem[i] = 8'h00; // zera tudo
    mem[0] = 8'b0000_1011; // LDA 11  <- opcode|operando
    mem[1] = 8'b0001_1100; // ADD 12
    // ...
    mem[10] = 8'b1111_0000; // HLT
    mem[11] = 8'd7;          //      (dados)
end
assign data_out = mem[addr]; // leitura assincrona (combinacional)
```



Endereço de 4 bits ⇒ 16 posições. Programa maior ⇒ menos espaço para dados.

Figura 4 — Os 16 slots compartilhados: programa (0-10) e dados (11-15) do Programa 2.

A figura mostra essa divisão para o Programa 2: as posições 0 a 10 guardam as instruções e 11 a 15 os dados. Fica evidente o compromisso imposto pelos 4 bits de endereço — programa e dados disputam os mesmos 16 slots, de modo que um programa maior deixa menos espaço para dados.

5.4. ULA combinacional em complemento de dois

A ULA é uma única expressão combinacional. A subtração usa o complemento de dois nativo do Verilog e o resultado de 8 bits satura por wraparound (módulo 256). Por ser combinacional, o resultado precisa

estabilizar dentro do estado T6 antes do posedge que o grava em A — o que é folgado no clock lento da placa. É essa largura fixa de 8 bits que limita o SAP-1 a valores de 0 a 255.

rtl/adder_subtractor.v

```
assign result = su ? (acc - breg) // Su = 1: subtrai (compl. de dois)
                : (acc + breg); // Su = 0: soma
```

5.5. HLT por clock-enable, não por gating de clock

Decisão de robustez central. Parar o processador desligando o clock por lógica combinacional (clock gating) é má prática em FPGA: introduz glitches no sinal de clock, viola as restrições de tempo (STA) e pode deixar registradores em estados inconsistentes. A solução adotada é registrar um flag halted (travado no estado T4, quando o opcode HLT já foi decodificado em T3) e usá-lo como clock-enable do contador de estados. Congelado o contador, a palavra de controle vira IDLE, nenhuma carga é ativada e todos os registradores seguram seus valores naturalmente — o processador "para" com o clock intacto e o estado preservado em T4. No topo, o clock do núcleo é ligado direto (wire clk_cpu = clk;), sem qualquer gating.

rtl/controller_sequencer.v – HALT registrado

```
reg halted;
wire run = ~halted; // clock-enable do contador de anel
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar) halted <= 1'b0;
    else if ((ring == T4) && (opcode == HLT)) halted <= 1'b1; // trava em T4
end
```

5.6. Estado ocioso de partida (IDLE) e recuperação de reset

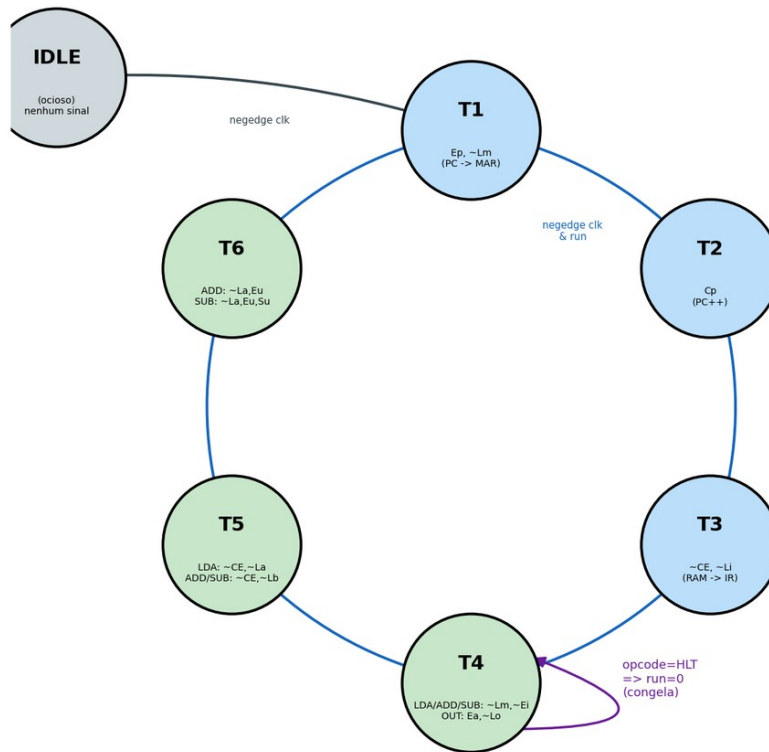
O contador inclui um estado de partida IDLE (nenhum T ativo, valor one-hot 000000). Ele absorve a fase do reset. Sem esse estado, o funcionamento correto dependeria de soltar o reset exatamente com o clock em nível conveniente — um problema clássico de recovery/removal do reset assíncrono. Com o IDLE, qualquer que seja a fase do clock ao soltar o reset, o primeiro negedge útil leva a máquina a T1 de forma determinística. Na camada de placa, esse cuidado é reforçado por um sincronizador de reset (assert assíncrono, desassert síncrono), descrito na seção 9.

6. Máquina de estados (unidade de controle)

6.1. Diagrama de estados (anel)

$\sim\text{CLR}=0$
(reset assinc.)

FSM do Controlador/Sequenciador - SAP-1



Azul = ciclo de BUSCA (T1-T3, igual p/ toda instrução) Verde = ciclo de EXECUÇÃO (T4-T6, depende do opcode)
Estado avança na borda de DESCIDA do clock; registradores carregam na SUBIDA.

Figura 5 — Contador de anel IDLE $\rightarrow$ T1 $\rightarrow \dots \rightarrow$ T6 $\rightarrow$ T1; laço roxo em T4 = congelamento por HLT.

6.2. Ciclo de execução por instrução

Ciclo de Execucao por Instrucao - FSM do SAP-1

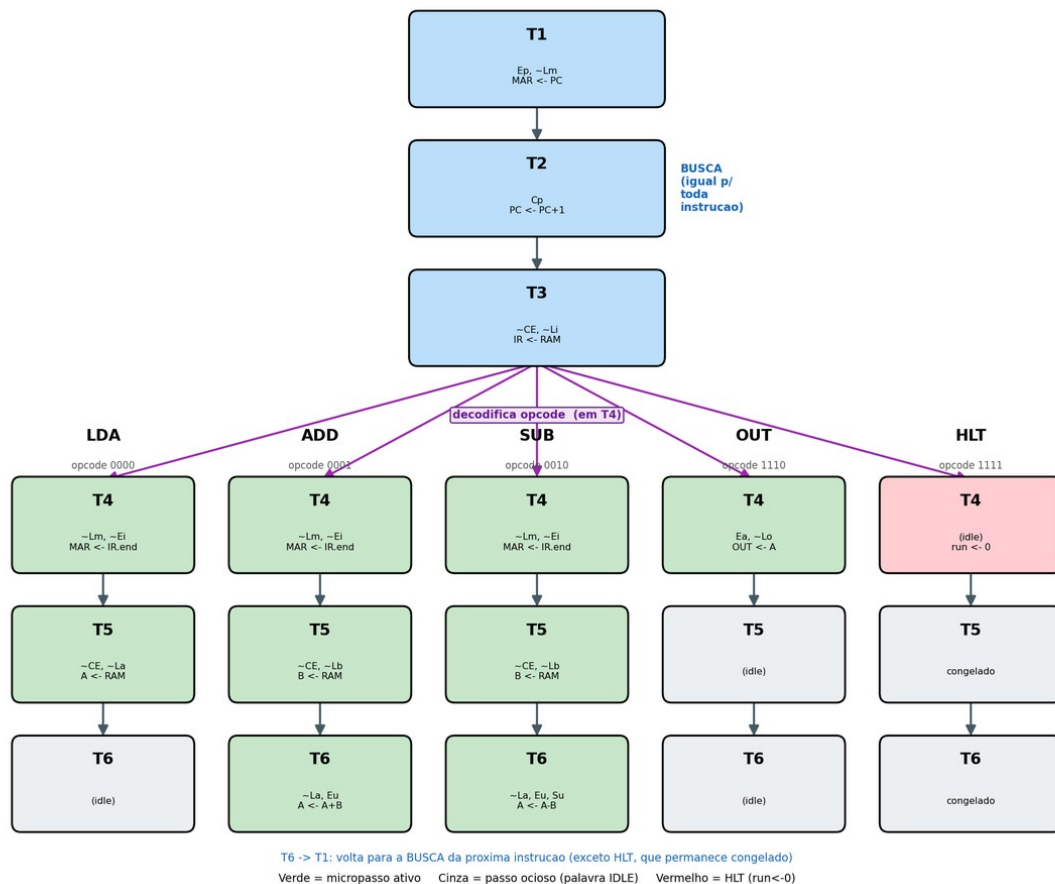


Figura 6 — Busca comum (T1-T3, azul) e ramificação da execução (T4-T6) por opcode.

A busca é idêntica para toda instrução; a diferença começa em T4, quando o opcode é decodificado. LDA usa até T5 ($A \leftarrow \text{RAM}$); ADD/SUB usam até T6 ($A \leftarrow A \pm B$); OUT resolve em T4; HLT congela em T4.

6.3. Codificação one-hot e a rotação do anel

O estado é mantido em one-hot de 6 bits (exatamente um bit em 1 por vez, salvo o IDLE que é todo-zero). One-hot foi escolhido por dois motivos concretos: a decodificação do estado vira um simples teste de um bit (barato e rápido em LUTs de FPGA), e a transição para o próximo estado é uma rotação de 1 bit — sem somador nem comparador. A rotação é feita por concatenação:

rotação one-hot (T6 → T1 fecha o anel)

```
// ring_r = {b5,b4,b3,b2,b1,b0} (T1=000001, T2=000010, ..., T6=100000)
ring_r <= {ring_r[4:0], ring_r[5]}; // desloca a esquerda e traz o bit 5 p/ o 0
// T1(000001)->T2(000010)->T3(000100)->T4(001000)->T5(010000)->T6(100000)->T1
```

O contador completo trata os quatro casos — reset, congelado (HLT), partida do IDLE e rotação — nesta ordem de prioridade:

rtl/controller_sequencer.v – contador de anel

```
always @(negedge clk or negedge clr_bar) begin
  if (!clr_bar)      ring_r <= IDLE_ST;           // reset -> IDLE
  else if (!run)     ring_r <= ring_r;           // HLT: congela
  else if (ring_r==IDLE_ST) ring_r <= T1;        // 1o negedge -> T1
  else               ring_r <= {ring_r[4:0], ring_r[5]}; // rotaciona
end
```

6.4. Duas implementações equivalentes do controle

A unidade de controle existe em dois arquivos, funcionalmente equivalentes (mesmas portas, mesma tabela de saída), como estudo comparativo:

Aspecto	controller_sequencer.v (usado)	controller_fsm.v (alternativo)
Estilo	contador de anel (shift register)	FSM clássica de 3 processos
Codificação do estado	one-hot (6 bits)	binária (3 bits, nomes simbólicos)
Próximo estado	rotação de 1 bit	case explícito S_T1→S_T2→...
Vantagem	decode/rotação triviais	leitura didática, estados nomeados

Trocar uma pela outra é só mudar o nome do módulo instanciado no topo (as portas são idênticas). O trecho de próximo-estado da versão clássica deixa a sequência explícita:

rtl/controller_fsm.v – lógica de próximo estado (2º processo)

```
always @(*) begin
  if (!run) next_state = state;           // HLT: congela
  else case (state)
    S_IDLE: next_state = S_T1;
    S_T1:  next_state = S_T2;   S_T2: next_state = S_T3;
    S_T3:  next_state = S_T4;   S_T4: next_state = S_T5;
    S_T5:  next_state = S_T6;   S_T6: next_state = S_T1; // fecha o anel
    default: next_state = S_IDLE;
  endcase
end
```

7. A palavra de controle de 12 bits

A cada estado a unidade de controle emite 12 bits (CON) que comandam todo o datapath. O layout é fixo (MSB → LSB):

```
CON = { Cp, Ep, ~Lm, ~CE, ~Li, ~Ei, ~La, Ea, Su, Eu, ~Lb, ~Lo }
bit:  11 10  9  8  7  6  5  4  3  2  1  0
```

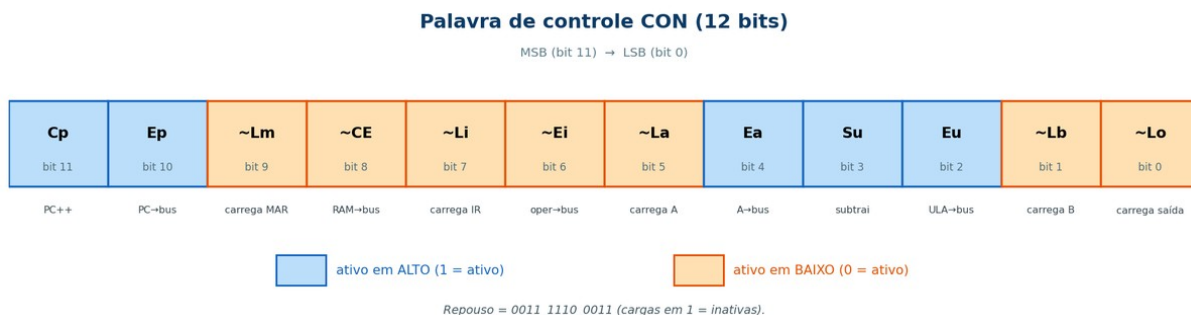


Figura 7 — Layout dos 12 bits de CON: azul = ativo em alto, laranja = ativo em baixo.

Na figura, os bits em azul são ativos em alto (habilitações) e os em laranja, ativos em baixo (cargas e a habilitação da RAM). Essa distinção visual explica por que a palavra de repouso mantém 1 nos bits de carga — em vez de zero — e serve de guia para ler a tabela detalhada a seguir.

Bit	Sinal	Ativo	Função quando ativo
11	Cp	alto	incrementa o PC
10	Ep	alto	PC escreve no barramento
9	~Lm	baixo	carrega o MAR a partir do barramento
8	~CE	baixo	RAM escreve no barramento
7	~Li	baixo	carrega o IR
6	~Ei	baixo	operando do IR no barramento
5	~La	baixo	carrega o acumulador
4	Ea	alto	acumulador escreve no barramento
3	Su	alto	ULA subtrai (0 = soma)
2	Eu	alto	ULA escreve no barramento
1	~Lb	baixo	carrega o registrador B
0	~Lo	baixo	carrega o registrador de saída

A tabela a seguir lista a palavra emitida em cada estado/opcode, os sinais que ficam ativos e a microoperação resultante. Os valores binários foram transcritos diretamente do código (e conferidos bit a bit):

Estado	Opcode	CON (12 bits)	Sinais ativos	Microoperação
IDLE	—	0011_1110_0011	(nenhum)	repouso
T1	todos	0101_1110_0011	Ep, ~Lm	MAR ← PC
T2	todos	1011_1110_0011	Cp	PC ← PC + 1
T3	todos	0010_0110_0011	~CE, ~Li	IR ← RAM[MAR]
T4	LDA/ADD/SUB	0001_1010_0011	~Lm, ~Ei	MAR ← IR.operando
T4	OUT	0011_1111_0010	Ea, ~Lo	saída ← A
T4	HLT	(IDLE)	run ← 0	congela em T4
T5	LDA	0010_1100_0011	~CE, ~La	A ← RAM[MAR]
T5	ADD/SUB	0010_1110_0001	~CE, ~Lb	B ← RAM[MAR]
T6	ADD	0011_1100_0111	~La, Eu	A ← A + B
T6	SUB	0011_1100_1111	~La, Eu, Su	A ← A – B

Estados não listados para um dado opcode (por exemplo, T5/T6 de OUT) emitem a palavra IDLE — são "vazios", o que explica por que toda instrução ocupa 6 estados mesmo quando usa menos.

8. Rastreamento ciclo a ciclo de uma instrução (ADD)

Para tornar concreta a interação controle × datapath, segue a execução de um ADD endereço, com o acumulador inicialmente em A e o dado D na posição apontada. Cada linha é um estado: o controle prepara a palavra (no negedge de entrada do estado) e a transferência se efetiva no posedge seguinte.

Estado	Sinais ativos	Quem fala no barramento	Quem carrega (posedge)
T1	Ep, ~Lm	PC	MAR ← PC
T2	Cp	(ninguém)	PC ← PC + 1
T3	~CE, ~Li	RAM	IR ← instrução ADD
T4	~Lm, ~Ei	operando do IR	MAR ← endereço do dado
T5	~CE, ~Lb	RAM	B ← D (o dado)

T6	$\sim La, Eu$	ULA (A + B)	$A \leftarrow A + D$
-----------	---------------	-------------	----------------------

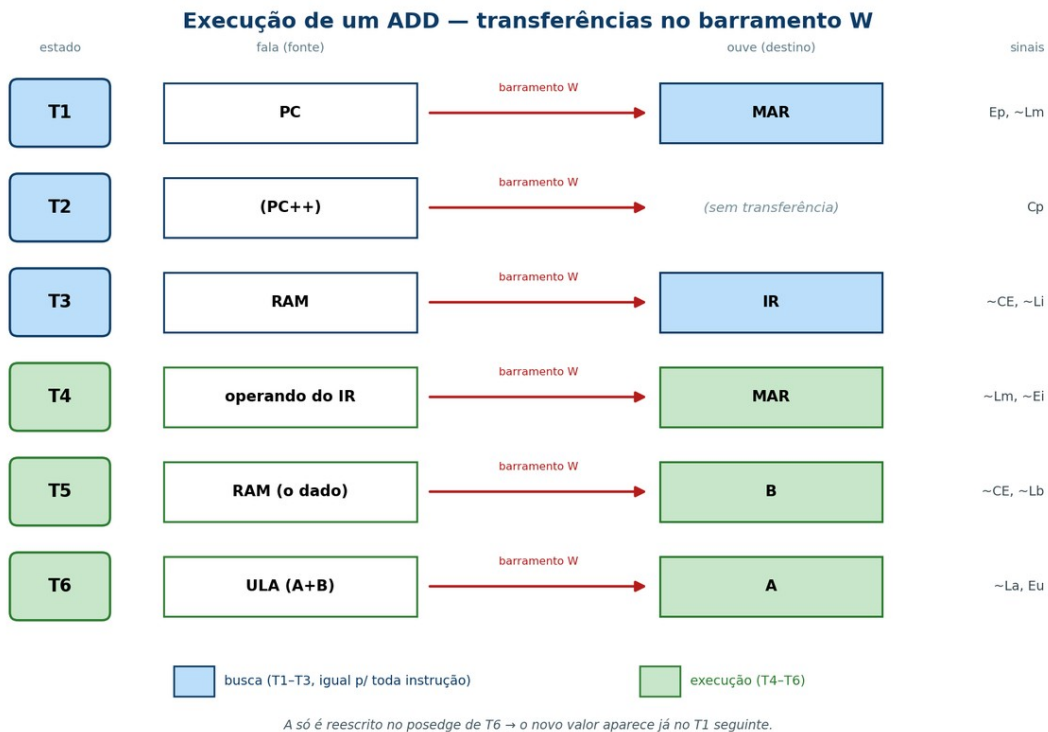


Figura 8 — Fluxo do ADD estado a estado: quem fala e quem ouve no barramento W.

A figura traduz a tabela acima em transferências pelo barramento: em cada estado, uma fonte (à esquerda) escreve no barramento W e um destino (à direita) o amostra. Vê-se que a busca (T1-T3, em azul) é idêntica à de qualquer instrução e que a identidade do ADD só aparece na execução (T4-T6, em verde), culminando em $A \leftarrow A + B$.

Observações. Em T1-T3 (busca) o processador nem sabe qual é a instrução — só em T3 o IR é carregado e o opcode passa a valer. Em T4 o operando (que é um **endereço**) é levado ao MAR; em T5 o dado lido vai para B; em T6 a ULA soma $A + B$ e o resultado volta para A. Como A só é escrito no posedge de T6, seu novo valor aparece "assentado" já no T1 da próxima instrução — exatamente o efeito de "um passo depois" visível nas ondas.

As outras instruções são variações deste esqueleto: LDA substitui o T5 por $A \leftarrow RAM$ (e não usa T6); SUB é igual ao ADD com $Su = 1$ em T6; OUT resolve em T4 (saída $\leftarrow A$) e deixa T5/T6 vazios; HLT apenas trava em T4.

9. Camada de placa (fpga/)

A pasta fpga adapta o núcleo à DE10-Lite sem tocar no rtl. O top de placa (sap1_fpga.v) reúne quatro cuidados de integração.

9.1. Geração e seleção de clock

O clock de 50 MHz é rápido demais para observar a execução. Um divisor gera ~1 Hz (um estado T por segundo), e um seletor permite alternar para clock manual passo a passo (um pulso por toque de botão), ideal para depurar e gravar vídeo:

rtl/./fpga/sap1_fpga.v – clocks

```
clock_divider #(.DIV(DIV)) u_div (.clk_in(clk50), .rst_n(rst_n), .clk_out(slow_clk));
debouncer     #(.N(500_000)) u_db (.clk(clk50), .rst_n(rst_n),
                                   .noisy(~KEY[1]), .clean(step_clk));
wire cpu_clk = SW[0] ? step_clk : slow_clk; // SW[0]: 0=auto(1Hz) 1>manual
```

O divisor conta $DIV-1$ e alterna a saída ($\text{freq} = 50 \text{ MHz} / (2 \cdot \text{DIV})$). O debouncer filtra o botão com um sincronizador de dois estágios seguido de um contador: só aceita o novo nível após ele permanecer estável por N ciclos (~10 ms), evitando múltiplos pulsos por toque.

9.2. Sincronizador de reset

O botão de reset é assíncrono ao clock. Para evitar violações de recovery/removal, o reset é aplicado de forma assíncrona (assert imediato) mas liberado de forma síncrona (desassert passa por dois flip-flops). Isso, somado ao estado IDLE do núcleo (seção 5.6), torna a partida totalmente determinística:

fpga/sap1_fpga.v – sincronizador de reset

```
always @(posedge clk50 or negedge ext_rst_n) begin
    if (!ext_rst_n) begin rs0 <= 0; rs1 <= 0; end // assert assíncrono
    else                begin rs0 <= 1; rs1 <= rs0; end // desassert sincronizado
end
wire rst_n = rs1; // ~CLR limpo para o núcleo
```

9.3. Mapeamento dos displays

Os seis displays de 7 segmentos mostram o funcionamento em tempo real. O opcode é traduzido para uma letra (em vez do hex cru) por um decodificador dedicado:

Display	Mostra	Fonte
HEX5	estado T atual (1-6; 0 = idle)	tstate → número
HEX4	instrução como letra: L A 5 o H	seg7_instr(opcode)
HEX3-HEX2	acumulador A (hex)	acc (registrador real)
HEX1-HEX0	resultado (registrador de saída, hex)	out_port
LEDR[9]	HLT (aceso ao parar)	hlt

fpga/seg7_instr.v – opcode → letra (segmentos ativos em baixo)


```

always @(*) case (opcode)
  4'b0000: seg = 7'b1000111; // L  (LDA)
  4'b0001: seg = 7'b0001000; // A  (ADD)
  4'b0010: seg = 7'b0010010; // 5  (SUB, ~ 'S')
  4'b1110: seg = 7'b0100011; // o  (OUT)
  4'b1111: seg = 7'b0001001; // H  (HLT)
  default: seg = 7'b1111111; // apagado
endcase

```

Há ainda um modo de depuração ($SW[1] = 1$) que redireciona os displays para mostrar o registrador B e o próprio barramento W a cada estado — útil para acompanhar as transferências internas na placa.

10. Código-fonte comentado dos módulos-chave

Antes do código, esta seção fixa os conceitos de Verilog necessários para lê-lo; em seguida, cada módulo é apresentado com o código e uma explicação das construções usadas.

10.1. Conceitos de Verilog usados

Verilog descreve hardware, não um programa que executa passo a passo. Dois estilos convivem: lógica combinacional (a saída segue as entradas continuamente, como um circuito de portas) e lógica sequencial (o valor só muda na borda do clock, formando registradores). As construções abaixo são as usadas neste projeto:

Construção	O que significa
module ... (portas)	define um bloco de hardware e suas conexões externas
input / output	direção de cada porta do módulo
wire	fio: valor contínuo, dirigido por assign ou por outro bloco; não guarda estado
reg	sinal atribuído dentro de um bloco always; vira registrador (se sequencial) ou lógica (se combinacional)
assign x = ...	atribuição contínua $\Rightarrow$ lógica combinacional (o fio acompanha a expressão o tempo todo)
always @(*)	bloco combinacional: reavalia quando qualquer entrada muda
always @(posedge clk)	bloco sequencial: age na borda de subida do clock $\Rightarrow$ gera flip-flops
... or negedge clr_bar	acrescenta o reset à lista de sensibilidade $\Rightarrow$ reset assíncrono
<= (não-bloqueante)	usado na lógica sequencial; as atribuições do

	bloco valem "todas juntas" na borda
= (bloqueante)	usado na lógica combinacional (always @(*))
localparam	constante nomeada (ex.: opcodes, códigos de estado)
case	seleção múltipla — usada para decodificar opcode/estado
{a, b}	concatenação de bits (ex.: estender 4→8 bits)
x[7:4]	fatiamiento: seleciona um intervalo de bits
cond ? a : b	operador ternário — um multiplexador de duas vias

Reset assíncrono: quando `clr_bar` aparece na lista de sensibilidade (`negedge clr_bar`), o registrador zera no instante em que o reset é ativado, sem esperar o clock — por isso o `if (!clr_bar)` vem sempre primeiro (prioridade).

Por que `<=` (não-bloqueante): na lógica sequencial, todas as atribuições de um bloco devem enxergar os valores antigos e mudar simultaneamente na borda. O `<=` garante isso e evita corridas; o `=` (bloqueante) fica reservado à lógica combinacional.

reg não é "registrador" por si só: `reg` apenas indica que o sinal é atribuído dentro de um `always`. Se o `always` for `@(posedge clk)`, a síntese cria um flip-flop; se for `@(*)`, cria lógica combinacional.

10.2. Program Counter

`rtl/program_counter.v`

```
module program_counter (
    input wire      clk,
    input wire      clr_bar, // ~CLR
    input wire      cp,      // Cp: habilita contagem
    output reg [3:0] pc_out
);
    always @(posedge clk or negedge clr_bar) begin
        if (!clr_bar) pc_out <= 4'b0000;
        else if (cp)  pc_out <= pc_out + 4'b0001;
    end
endmodule
```

Leitura do código: `pc_out` é output reg [3:0] — quatro bits (0 a 15) atribuídos dentro do `always`, logo é um registrador. O bloco reage a duas bordas: a subida do clock e a descida de `clr_bar`. A ordem das condições define a prioridade: primeiro `if (!clr_bar)` zera (reset assíncrono); senão, na borda de clock, se `cp = 1`, soma 1 (contagem); se `cp = 0`, não há atribuição e o valor é mantido — a síntese infere um flip-flop com enable. Ao passar de 15, os 4 bits voltam a 0 naturalmente (wraparound).

10.3. Registrador de Instrução (decodifica opcode | operando)

rtl/instruction_register.v

```
module instruction_register (
    input wire clk, clr_bar, li_bar,
    input wire [7:0] bus_in,
    output wire [3:0] opcode,    // nibble alto
    output wire [3:0] operand    // nibble baixo
);
    reg [7:0] ir;
    always @(posedge clk or negedge clr_bar) begin
        if (!clr_bar) ir <= 8'b0;
        else if (!li_bar) ir <= bus_in;
    end
    assign opcode = ir[7:4];
    assign operand = ir[3:0];
endmodule
```

Leitura do código: o registrador interno ir (8 bits) segue o mesmo padrão do PC — carrega bus_in quando li_bar = 0, senão mantém. A parte nova são os dois assign: como são atribuições contínuas, opcode e operand são wire e refletem sempre o conteúdo de ir. O fatiamento ir[7:4] pega os quatro bits altos (a instrução) e ir[3:0] os quatro baixos (o operando/endereço). Ou seja, a "decodificação" em dois campos não custa nenhum circuito extra — é apenas fiação para as duas metades do registrador.

MAR, registrador B e registrador de saída seguem exatamente o padrão do acumulador (seção 5.2), mudando apenas a largura (MAR tem 4 bits) e o nome da carga.

10.4. Integração no topo (instanciação)

rtl/sap1_top.v – trecho de instanciação

```
wire clk_cpu = clk;    // clock unico, SEM gating (HLT usa clock-enable)

accumulator u_acc (.clk(clk_cpu), .clr_bar(clr_bar), .la_bar(la_bar),
    .bus_in(w_bus), .acc_out(acc_out));
adder_subtractor u_alu (.su(su), .acc(acc_out), .breg(b_out), .result(alu_out));
controller_sequencer u_ctrl (.clk(clk_cpu), .clr_bar(clr_bar), .opcode(ir_opcode),
    .cp(cp), .ep(ep), .lm_bar(lm_bar), .ce_bar(ce_bar), .li_bar(li_bar),
    .ei_bar(ei_bar), .la_bar(la_bar), .ea(ea), .su(su), .eu(eu),
    .lb_bar(lb_bar), .lo_bar(lo_bar), .con(con), .ring(ring), .hlt(hlt));
```

Leitura do código: instanciar um módulo é criar uma cópia física do bloco e ligar seus pinos. A forma nome_do_modulo nome_da_instancia (.porta(sinal), ...) faz a conexão por nome — por exemplo, .bus_in(w_bus) liga a entrada bus_in do acumulador ao barramento. O mesmo w_bus chega a vários blocos ao mesmo tempo (todos podem ler), mas só um escreve por vez, graças ao mux. Note que clk_cpu é apenas o clock de entrada sem nenhuma porta lógica no caminho — coerente com a decisão de não fazer gating (o HLT congela via clock-enable, não desligando o clock).

11. Conclusão

A implementação seguiu três eixos. Modularidade: um arquivo por bloco, com o topo apenas interligando-os pelo barramento, e uma separação estrita rtl x fpga. Boas práticas de FPGA no nível de hardware: barramento por multiplexador (sem tri-state), reset assíncrono com sincronizador de desassert, ausência total de gating de clock (HLT por clock-enable) e disciplina de duas bordas para eliminar corrida entre controle e dados.

Clareza didática: palavra de controle transcrita da tabela de microoperações, máquina de estados explícita e disponível em duas formas equivalentes (anel one-hot e FSM clássica). O resultado é um núcleo sintetizável, determinístico na partida e legível — cuja unidade de controle, o ponto mais delicado, está descrita de forma direta e verificável. A validação por formas de onda consta do relatório de verificação por simulação.